

AFF Global CSS Class Management — Implementation Plan

Document version: 2.0

Date: 2026-05-21

Based on: Direct source inspection of Elementor plugin (site: [claude-wordpress-integration-novamira](#)) + full read of existing AFF codebase

Supersedes: [EFF-Phase3-Classes-Plan.md](#) (v1.0, 2026-04-03)

Status: Ready for implementation — all open questions from v1.0 resolved

Table of Contents

1. [What Elementor Global Classes Are](#)
 2. [Storage Format — Confirmed from Source](#)
 3. [Read Path](#)
 4. [Write Path](#)
 5. [AFF Data Model](#)
 6. [UI Design](#)
 7. [PHP Implementation Plan](#)
 8. [JS Implementation Plan](#)
 9. [CSS Implementation Plan](#)
 10. [Phased Delivery](#)
 11. [Open Questions](#)
-

1. What Elementor Global Classes Are

Global Classes are Elementor V4's reusable CSS class system. Each class:

- Has a user-defined name (the **label** field) that becomes the literal CSS class name on the element — e.g. a class labelled **primary-btn** renders as `class="primary-btn"` in the HTML DOM
- Contains one or more **variants** — each variant is a set of CSS property values scoped to a specific breakpoint and/or pseudo-state (hover, active, focus)
- Is referenced from Elementor atomic widget elements by an opaque internal ID (**g-XXXXXXX**) rather than by label; the ID-to-label mapping happens during render via `Atomic_Global_Styles::transform_classes_names()`
- Has a site-wide usage count accessible via a separate REST endpoint

What the CSS output looks like:

Elementor V4 renders global class CSS via `Atomic_Global_Styles`, not the kit CSS file. The CSS is injected via `Atomic_Styles_Manager` during page render as an inline style block or a generated file. The selector uses the **label** as the class name, prefixed by `.elementor` (the **selector_prefix** in `Styles_Renderer`). Example:

```
.elementor .primary-btn { color: #ff0000; font-size: 16px; }
.elementor .primary-btn:hover { color: #cc0000; }
@media(max-width:1024px) { .elementor .primary-btn { font-size: 14px; } }
```

Source: `modules/atomic-widgets/styles/styles-renderer.php`, lines 91–113.

Global Classes vs Variables: Variables are CSS custom properties in the `:root` block of the kit CSS file — AFF reads these via `AFF_CSS_Parser`. Global Classes are completely separate: stored in a different meta key, served via a different REST endpoint, rendered via a different CSS pipeline. AFF's existing CSS parser never sees class definitions.

2. Storage Format — Confirmed from Source

2.1 Post Meta Keys

Source: `modules/global-classes/global-classes-repository.php`, lines 15–16.

```
const META_KEY_FRONTEND = '_elementor_global_classes';
const META_KEY_PREVIEW  = '_elementor_global_classes_preview';
```

Both keys are on the active kit post (the same post AFF already reads for variables via `get_option('elementor_active_kit')`).

The meta value is stored via `Kit::update_json_meta()` / `Kit::get_json_meta()` — Elementor's own wrapper around `update_post_meta` that JSON-encodes the value. When read raw via `get_post_meta()`, it may be a JSON string or a PHP array depending on WordPress's auto-unserialization. The repository uses `get_json_meta()` which always returns a PHP array.

2.2 Storage Structure

The meta value is a PHP array with exactly two keys:

```
{
  "items": {
    "g-19ae5e7": {
      "id": "g-19ae5e7",
      "type": "class",
      "label": "primary-btn",
      "variants": [ ... ]
    },
    "g-8e879b6": {
      "id": "g-8e879b6",
      "type": "class",
      "label": "hero-heading",
      "variants": []
    }
  }
}
```

```
    },
    "order": [ "g-8e879b6", "g-19ae5e7" ]
  }
}
```

Note: The v1.0 plan documented the REST response as { data: {}, meta: { order: [] } }. That was the REST API response shape. The actual stored meta shape is { items: {}, order: [] }. The REST API wraps items as data and order as meta.order in its response. AFF must read items and order from the meta or REST response accordingly.

Source: global-classes-repository.php line 58: \$all['order'] = Global_Classes_Parser::sanitize_order(\$all['items'] ?? [], \$all['order'] ?? []); — confirms items and order as the top-level keys.

2.3 Item Structure — Confirmed from Source

Each item in items has these fields, validated by Global_Classes_Parser::parse_items() via Style_Parser::parse():

Field	Type	Description
id	string	Opaque ID, g- prefix + 7 hex chars (e.g. g-19ae5e7)
type	"class"	Always "class" for Global Classes
label	string	The CSS class name — appears literally in HTML DOM; max 50 chars
variants	array	Per-breakpoint/state style objects (see below); may be empty

Optional field confirmed from design-system-sync/classes/classes-provider.php line 39:

Field	Type	Description
sync_to_v3	bool	If true, class is synced to Elementor V3 typography globals

2.4 Variant Structure — Confirmed from Source

Source: modules/atomic-widgets/styles/styles-renderer.php, lines 40–86, and modules/design-system-sync/classes/classes-provider.php, lines 58–85.

Each variant in the variants array:

```
{
  "props": {
    "color": { "$$type": "color", "value": "#ff0000" },
    "font-size": { "$$type": "size", "value": { "size": 16, "unit": "px" } },
    "padding": { "$$type": "dimensions", "value": { "top": ..., "right": ..., ...
  } }
},
"meta": {
  "breakpoint": "mobile",
  "state": "hover"
}
```

```

    },
    "custom_css": {
        "raw": ""
    }
}

```

- **meta.breakpoint**: string key (e.g. "mobile", "tablet") or **null** for desktop
- **meta.state**: string (e.g. "hover", "active", "focus") or **null** / "normal" for the default state
- **props**: keyed by CSS property name; values use Elementor's **\$\$type** prop format (same format used for variables in **_elementor_global_variables**)
- **custom_css.raw**: arbitrary CSS string injected after the resolved props

The full set of supported CSS properties is defined in **modules/atomic-widgets/styles/style-schema.php**. It covers layout, typography, spacing, border, background, effects, and alignment — approximately 40+ property types.

2.5 REST API Structure

Endpoint: **wp-json/elementor/v1/global-classes**

Methods: GET, PUT

Permission (GET): **is_user_logged_in()**

Permission (PUT): **current_user_can('elementor_global_classes_update_class')** — a custom capability added by **Add_Capabilities** migration (administrator role only by default)

Source: **global-classes-rest-api.php**, lines 36–53 (GET) and 73–145 (PUT), and **database/migrations/add-capabilities.php**.

The GET response wraps the meta **items** as **data** and **order** as **meta.order**:

```

{
  "data": { "g-19ae5e7": { ... }, "g-8e879b6": { ... } },
  "meta": { "order": [ "g-8e879b6", "g-19ae5e7" ] }
}

```

Source: **global-classes-rest-api.php**, lines 148–155 — **Response_Builder::make((object) \$classes->get_items()->all())->set_meta(['order' => \$classes->get_order()->all()])**.

Class limit: **const MAX_ITEMS = 100** (source: **global-classes-rest-api.php** line 18). The earlier 50-class limit from Alpha has been raised to 100 in the current source.

2.6 Feature Flag

Global Classes require **two** Elementor experiments to be active:

1. **e_classes** (experiment name 'e_classes', constant **Module::NAME**)
2. **e_atomic_elements** (Atomic Widgets experiment)

Source: **modules/global-classes/module.php**, lines 41–43. If either is inactive, no global classes infrastructure is registered and the REST endpoint will not exist.

AFF must check for this and surface a clear message if the endpoint returns 404.

3. Read Path

AFF reads class definitions via one of two methods, with automatic fallback.

3.1 Primary: Direct Post Meta Read

This is faster and more reliable on local environments than an HTTP loopback.

```
$kit_id = AFF_CSS_Parser::get_active_kit_id(); // reuse existing static method
$raw    = get_post_meta( $kit_id, '_elementor_global_classes', true );
```

The raw value will be a JSON string (WordPress stores it as a serialized/encoded string via Elementor's `update_json_meta`). Decode with `json_decode($raw, true)`. The decoded structure is `{ items: {}, order: [] }`.

Do NOT use `get_json_meta()` — that is Elementor's internal Kit document method and requires Elementor's DI container to be fully initialized, which may not be the case during an AJAX call outside the Elementor editor context.

3.2 Fallback: REST API

If the post meta is empty or unavailable, AFF falls back to the REST endpoint using `wp_remote_get()`:

```
$url      = rest_url( 'elementor/v1/global-classes' );
$nonce    = wp_create_nonce( 'wp_rest' );
$response = wp_remote_get( $url, [
    'headers' => [ 'X-WP-Nonce' => $nonce ],
    'timeout' => 10,
] );
```

The REST response shape differs from the meta shape: `{ data: {}, meta: { order: [] } }`. The normalization method must handle both shapes.

3.3 Normalization

The normalization method translates the raw Elementor structure into AFF class objects. It handles both the direct meta shape (`items/order`) and the REST shape (`data/meta.order`):

```
$items = $raw['items'] ?? $raw['data'] ?? [];
$order = $raw['order'] ?? $raw['meta']['order'] ?? [];
```

3.4 What AFF Does NOT Read

AFF does not parse variant CSS property values in the initial phases. The `variants` array is stored as-is in the AFF data model for potential future use, but AFF's list view only uses `count(variants) > 0` to set `has_styles`. Full variant parsing (for a future "preview CSS" feature) is deferred to Phase 4.

4. Write Path

4.1 Scope of Writes

AFF writes back to Elementor for two operations:

1. **Sync (read-only from Elementor's perspective):** AFF fetches the current state from Elementor and updates its local store. No write to Elementor.
2. **Class creation / rename / delete:** AFF writes back via the REST PUT endpoint. This is the only safe write path — the PUT endpoint runs Elementor's own validation (`Global_Classes_Parser::parse_items()`) and resolver (`Global_Classes_Changes_Resolver`).

AFF does not write CSS property values to Elementor. That remains the Elementor editor's job. AFF can create empty classes (no variants) and rename/delete existing ones.

4.2 Write via REST PUT

Endpoint: `PUT wp-json/elementor/v1/global-classes`

Required capability: `elementor_global_classes_update_class` (administrator only)

The PUT body requires:

```
{
  "context": "frontend",
  "changes": {
    "added": ["g-newid1"],
    "deleted": ["g-oldid1"],
    "modified": ["g-modid1"]
  },
  "items": {
    "g-newid1": { "id": "g-newid1", "type": "class", "label": "my-class",
"variants": [] },
    "g-modid1": { "id": "g-modid1", "type": "class", "label": "renamed",
"variants": [...] }
  },
  "order": ["g-newid1", "g-modid1"]
}
```

The `changes` object drives the resolver (`Global_Classes_Changes_Resolver`): items in `added` and `modified` are merged into the stored set; items in `deleted` are removed. Items not listed in `changes` are preserved verbatim from the stored set.

ID generation for new classes: New class IDs must match the `g-XXXXXXX` format (7 hex chars). AFF generates them server-side: `'g-' . substr(bin2hex(random_bytes(4)), 0, 7)`.

Duplicate label handling: If a new class label duplicates an existing one, Elementor prefixes it with `DUP_` and returns a `DUPLICATED_LABEL` code in the response body (not a 4xx error — still 200). AFF must detect this and surface the renamed label to the user.

Source: `global-classes-rest-api.php`, lines 184–241.

4.3 Nonce for Write Calls

The PUT endpoint uses the standard WordPress REST nonce (`wp_rest` action). AFF generates it server-side via `wp_create_nonce('wp_rest')` and passes it in the `X-WP-Nonce` header from the PHP AJAX handler's `wp_remote_request()` call.

5. AFF Data Model

5.1 AFF.state Integration

`AFF.state.classes` already exists in `aff-app.js` line 18 and `class-aff-data-store.php` line 38. The array holds AFF class objects. No new top-level state key is needed.

5.2 AFF Class Object

```
{
  "id": "uuid-v4",
  "elementor_id": "g-19ae5e7",
  "name": "primary-btn",
  "label": "primary-btn",
  "type": "global",
  "source": "elementor-fetched",
  "status": "synced",
  "group": "Classes",
  "category": "Buttons",
  "category_id": "uuid-of-aff-category",
  "order": 0,
  "has_styles": true,
  "notes": "",
  "tags": [],
  "created_at": "2026-05-21T10:00:00+00:00",
  "updated_at": "2026-05-21T10:00:00+00:00",
  "last_synced_at": "2026-05-21T10:00:00+00:00"
}
```

Field notes:

- `elementor_id`: The `g-XXXXXXX` ID from Elementor — stored verbatim. Used for all write operations back to Elementor.
- `name` and `label`: Both are initialized from Elementor's `label` field. `label` is the AFF display label (user-overridable). `name` stays fixed as the CSS class identifier.
- `has_styles`: `count($item['variants']) > 0`. Not a full parse — just a non-empty check.
- `status` enum: `synced` | `modified` | `aff-only` | `orphaned`

- **synced**: matches last Elementor fetch
- **modified**: AFF-side metadata changed (notes, tags, category) since last sync
- **aff-only**: present in AFF file but not returned by current Elementor fetch (class was deleted in Elementor)
- **orphaned**: present in Elementor but not yet in AFF file (appeared since last sync — resolved by syncing)
- Fields dropped from v1.0 plan: **has_responsive**, **states**, **breakpoints**, **usage_count**. These can be derived from **variants** data on demand; storing them as separate scalar fields creates sync drift. **usage_count** is available via a separate REST endpoint (**/usage**) and should be fetched on demand rather than stored.

5.3 **.aff.json** Storage

Classes live in the top-level **classes** array. Class categories live in **config.classCategories**.

```
{
  "version": "1.0",
  "config": {
    "categories": [...],
    "fontCategories": [...],
    "numberCategories": [...],
    "classCategories": [
      { "id": "uuid", "name": "Buttons", "order": 0, "locked": false },
      { "id": "uuid", "name": "Typography", "order": 1, "locked": false },
      { "id": "uuid", "name": "Uncategorized", "order": 999, "locked": true }
    ]
  },
  "variables": [...],
  "classes": [
    { "id": "...", "elementor_id": "g-19ae5e7", "name": "primary-btn", ... }
  ],
  "components": []
}
```

5.4 Relationship to Variable Model

Aspect	Variables	Classes
Elementor storage	_elementor_global_variables meta key	_elementor_global_classes meta key
AFF read source	CSS file (primary) + post meta	Post meta (primary) + REST fallback
CSS output	:root custom properties	Per-class rule blocks (Atomic Styles pipeline)
AFF write back	REST + direct meta (aff_commit_to_elementor)	REST PUT only (no direct meta write)

Aspect	Variables	Classes
Category system	<code>config.categories</code> / <code>fontCategories</code> / <code>numberCategories</code>	<code>config.classCategories</code>
Value editing in AFF	Yes (color picker, number/unit fields)	No — CSS properties stay in Elementor editor

6. UI Design

6.1 Left Panel

The **Classes** node already exists in the left panel as a fixed item. Phase implementation expands it:

▼ Classes

Buttons (3)

Typography (7)

Layout (2)

Uncategorized (1)

← fixed, click to expand/collapse

← category + count badge

← locked, always last

Category items follow identical interaction rules as variable categories: click to load, keyboard nav (Tab/arrow), active highlight with `--aff-clr-accent`. Count badge shows non-deleted classes in that category.

File to modify: `admin/js/aff-panel-left.js` — add `renderClassesTree(categories, classCounts)` following the existing variable tree renderer.

6.2 Edit Space — Class List View

When a category is selected, the center Edit Space shows a table of classes:

≡ primary-btn	Buttons	● synced	has styles	[:]
≡ secondary-btn	Buttons	● synced	no styles	[:]
≡ hero-heading	Typography	⦿ aff-only	has styles	[:]

Columns:

Column	Content	Notes
Drag handle ≡	<code>.aff-drag-handle</code>	Drag-to-reorder within category
Class name	<code>name</code> in monospace	Read-only — editing opens the detail modal
Category	Badge	Shows current category

Column	Content	Notes
Status	Color dot + label	synced (green), modified (gold), aff-only (muted), orphaned (red/orange)
Styles indicator	"has styles" / "no styles"	Derived from has_styles
Actions [:]	Dropdown	Edit notes, Move to category, Rename (→ Elementor), Delete (→ Elementor), Open in Elementor

Important differences from Variable rows:

- No inline value editing (no color picker, no text input for CSS props)
- No "value" column
- The "Open in Elementor" action links to the Elementor editor (no deep link to Class Manager is available — link to editor root)
- Rename and Delete actions commit to Elementor via the REST PUT endpoint, not just to the AFF file

6.3 Detail Modal

Clicking a class name opens the detail modal via AFF.Modal.open():

primary-btn[×]

CSS class: primary-btn

Elementor ID: g-19ae5e7

Label: [Primary Button]

Category: [Buttons ▼]

Status:

● synced (last synced 2026-05-21)

Styles defined: Yes (3 variants)

Tags: [button] [cta] [+ add tag]

Notes:

Primary CTA. Gold accent background.

[Save]

[Open Elementor →]

Editable fields: label (AFF display name), category (dropdown of classCategories), tags, notes.
Read-only: CSS class name, Elementor ID, status, styles indicator.
"Open Elementor" is a plain link to the WP admin Elementor editor URL — no deep link to Class Manager.

6.4 Toolbar — Sync Button

A "Sync Classes" button in the toolbar triggers `aff_sync_classes` AJAX action. This is the primary way classes enter the AFF system.

Sync behavior:

1. Fetch from Elementor (post meta primary, REST fallback)
2. Merge with existing AFF class store (non-destructive: AFF-only metadata preserved)
3. Mark classes missing from the Elementor response as `aff-only`
4. Add classes newly found in Elementor that are absent from AFF as `orphaned` (auto-resolved to `synced` on the same sync)
5. Return updated class list and counts to JS

6.5 Empty State

If no classes are found in Elementor:

No Global Classes found in Elementor.

Create classes in the Elementor Class Manager, then sync here.

[Sync Classes] [Open Elementor →]

6.6 Create New Class (Phase 3)

AFF can create empty classes in Elementor. A "New Class" button opens a modal asking for the class name (label). On confirm:

1. AFF generates a `g-XXXXXXX` ID
2. AJAX handler calls the REST PUT endpoint with `changes.added = [newId]` and a new item entry with empty `variants`
3. On success, the new class is added to AFF's local store with `source: 'user-created'`
4. User is directed to the Elementor editor to add styles

Label validation client-side: alphanumeric, hyphens, underscores only; no leading digit; max 50 chars.

7. PHP Implementation Plan

7.1 New File: `includes/class-aff-classes-reader.php`

Read-only class. Parallel to `AFF_CSS_Parser` but for a different data source.

Key methods:

```
class AFF_Classes_Reader {  
  
    // Primary: read directly from post meta. Fast, no HTTP.  
    public function read_from_postmeta(): array { ... }  
}
```

```

// Fallback: fetch via Elementor REST endpoint.
public function fetch_from_rest(): array { ... }

// Normalize either meta shape { items, order } or REST shape { data,
meta.order }
// into a flat array of AFF class objects.
public function normalize( array $raw ): array { ... }

// Entry point: try postmeta first, fall through to REST.
public function get_all(): array {
    $result = $this->read_from_postmeta();
    if ( empty( $result ) ) {
        $result = $this->fetch_from_rest();
    }
    return $result;
}
}

```

`read_from_postmeta()` implementation:

```

$kit_id = AFF_CSS_Parser::get_active_kit_id();
$raw     = get_post_meta( $kit_id, '_elementor_global_classes', true );
// $raw is a JSON string – decode it
if ( is_string( $raw ) ) { $raw = json_decode( $raw, true ); }
return $this->normalize( $raw ?: [] );

```

`normalize()` handles both response shapes:

```

$items = $raw['items'] ?? $raw['data'] ?? [];
$order = $raw['order'] ?? $raw['meta']['order'] ?? [];
// Build order-index lookup, iterate $items, return AFF class objects

```

Rules for this class:

- `if ( ! defined( 'ABSPATH' ) ) { exit; }` at top
- `AFF_` prefix
- Read-only — never writes to any WordPress data
- No WordPress dependencies in `normalize()` itself

7.2 New File: `includes/class-aff-classes-writer.php`

Handles writing changes back to Elementor via the REST PUT endpoint.

```

class AFF_Classes_Writer {

    // Create a new empty class in Elementor. Returns the new g-XXXXXXX ID or null
    on failure.

```

```

    public function create( string $label ): ?string { ... }

    // Rename an existing class by elementor_id.
    public function rename( string $elementor_id, string $new_label ): bool { ...
}

    // Delete a class by elementor_id.
    public function delete( string $elementor_id ): bool { ... }

    // Send a PUT request with the given changes payload.
    private function put( array $payload ): array { ... }

    // Generate a g-XXXXXXX style ID.
    private function generate_elementor_id(): string {
        return 'g-' . substr( bin2hex( random_bytes( 4 ) ), 0, 7 );
    }
}

```

Each method fetches the current full class list first (via `AFF_Classes_Reader::read_from_postmeta()`), constructs the `changes + items + order` payload, and calls `put()`. This fetch-then-modify pattern is required because the PUT endpoint is a full-replace operation for the touched items only.

Authentication: `wp_create_nonce('wp_rest')` — requires the request to run as a logged-in user with `elementor_global_classes_update_class` capability (administrator). AFF AJAX handlers run in the WP context, so `wp_create_nonce()` works without special setup.

7.3 Modifications to `class-aff-data-store.php`

7.3.1 `subgroup_to_cat_key()` — add Classes entry

```

$map = array(
    'Colors'   => 'categories',
    'Fonts'    => 'fontCategories',
    'Numbers'  => 'numberCategories',
    'Classes' => 'classCategories',    // ← add this
);

```

This gives Classes free access to all five existing `*_for_subgroup()` methods.

7.3.2 Classes CRUD methods

Add to the Classes CRUD section (currently only `get_classes()`):

```

public function add_class( array $class ): string { ... }    // returns UUID
public function update_class( string $id, array $data ): bool { ... }
public function delete_class( string $id ): bool { ... }
public function find_class_by_elementor_id( string $el_id ): ?array { ... }
public function import_fetched_classes( array $fetched ): array { ... } //
returns ['added', 'updated', 'orphaned']

```

7.3.3 `class_defaults()` private helper

```
private function class_defaults(): array {
    return array(
        'id'                => '',
        'elementor_id'      => null,
        'name'              => '',
        'label'             => '',
        'type'              => 'global',
        'source'            => 'elementor-fetched',
        'status'            => 'synced',
        'group'             => 'Classes',
        'category'          => 'Uncategorized',
        'category_id'       => '',
        'order'             => 0,
        'has_styles'        => false,
        'notes'             => '',
        'tags'              => [],
        'created_at'        => '',
        'updated_at'        => '',
        'last_synced_at'    => '',
    );
}
```

7.3.4 `migrate_data()` extension

Add a classes migration block after the variables block to backfill new fields on older `.aff.json` files.

7.3.5 `delete_category_for_subgroup()` — Classes branch

The existing method moves variables to Uncategorized when a category is deleted. Add a parallel branch for **Classes**: when a class category is deleted with `delete_vars = false`, clear `category_id` and `category` on affected class objects in `$this->data['classes']`.

7.4 Modifications to `class-aff-ajax-handler.php`

Add new AJAX actions to the `$actions` array in `register_handlers()`:

```
// Phase 3 – Classes endpoints
'aff_sync_classes',
'aff_get_classes',
'aff_update_class_meta',
'aff_move_class_category',
'aff_create_class',
'aff_rename_class',
'aff_delete_class',
// Class category management (reuse same pattern as variable categories)
'aff_save_class_category',
```

```
'aff_delete_class_category',  
'aff_reorder_class_categories',
```

Key endpoint signatures:

Action	Input	Output
aff_sync_classes	filename	Updated class list + counts
aff_get_classes	filename	Class list from AFF store only
aff_update_class_meta	filename, class_id, data (JSON: label/notes/tags/category/category_id)	Updated class object
aff_create_class	filename, label	New class object with elementor_id
aff_rename_class	filename, class_id, new_label	Updated class object
aff_delete_class	filename, class_id	Success/failure
aff_save_class_category	filename, category (JSON)	Category list
aff_delete_class_category	filename, category_id, delete_classes	Updated class list + categories
aff_reorder_class_categories	filename, ordered_ids (JSON)	Success

All handlers call `$this->verify_request()` and `manage_options` capability check — identical to existing handlers.

7.5 Modifications to `class-aff-loader.php`

Register the two new PHP classes:

```
require_once AFF_PLUGIN_DIR . 'includes/class-aff-classes-reader.php';  
require_once AFF_PLUGIN_DIR . 'includes/class-aff-classes-writer.php';
```

8. JS Implementation Plan

8.1 New File: `admin/js/aff-classes.js`

A new module following the `AFF.Variables` factory pattern, but simpler (no expand panel, no color picker, no inline value editing).

```
AFF.Classes = {  
  _collapsedIds: {},
```

```

    _cfg: {
      setName: 'Classes',
      catKey: 'classCategories',
    },

    init: function () { ... },
    loadClasses: function (selection) { ... },
    _renderView: function (category, classes) { ... },
    _renderClassRow: function (cls) { ... },
    _openDetailModal: function (classId) { ... },
    _syncClasses: function () { ... },
    _createClass: function () { ... },
    _renameClass: function (classId, newLabel) { ... },
    _deleteClass: function (classId) { ... },
  };

```

Intercepting `EditSpace.loadCategory`: Same pattern as `AFF.Variables.initSet()` — wrap `AFF.EditSpace.loadCategory` and handle calls where `selection.group === 'Classes'`.

Reusing `AFF.CatMixin`: Apply `Object.assign(AFF.Classes, AFF.CatMixin)` at the end of `aff-app.js`'s `Object.assign` block. `AFF.CatMixin` provides `_addCategory`, `_deleteCategory`, `_saveCategoryName`, `_duplicateCategory`, `_ajaxReorderCategories` — all free.

Reusing `AFF.VarDrag`: The drag-and-drop system works on `.aff-color-row` elements with `data-var-id`. `AFF.Classes` rows use the same element structure and same drag init pattern. The only difference is the AJAX action for persisting reorder (`aff_update_class_meta` with the new order values).

Reusing `AFF.Modal`: All modals (detail view, confirm delete, add category) use the existing `AFF.Modal.open()`. No new modal infrastructure needed.

Row data attributes:

```

<div class="aff-color-row aff-class-row" data-var-id="{cls.id}" data-elementor-
id="{cls.elementor_id}">
  <div class="aff-drag-handle">≡</div>
  <span class="aff-class-name">{cls.name}</span>
  <span class="aff-class-category-badge">{cls.category}</span>
  <span class="aff-status-dot" data-status="{cls.status}"></span>
  <span class="aff-class-styles-flag">{has_styles}</span>
  <button class="aff-class-actions" data-action="open-menu"></button>
</div>

```

Using `.aff-color-row` as the base class means `AFF.VarDrag` picks it up without changes.

8.2 Modifications to `aff-app.js`

- Add `Object.assign(AFF.Classes, AFF.CatMixin)` alongside the existing `Object.assign` calls at the bottom of the shared mixin block (around line 808)

- Add `AFF.Classes.init()` call in the `DOMContentLoaded` block, after `AFF.Variables.initSet()` calls
- Add `findClassById(id)` and `findClassByKey(key)` to `AFF.Utills` (parallel to existing `findVarById` / `findVarByKey`)

AFF.Utills additions:

```
findClassById: function (id) {
    var classes = AFF.state.classes || [];
    for (var i = 0; i < classes.length; i++) {
        if (classes[i].id === id) { return classes[i]; }
    }
    return null;
},

getClassesForCategoryId: function (catId) {
    return (AFF.state.classes || []).filter(function (c) {
        return c.category_id === catId && c.status !== 'deleted';
    });
},
```

8.3 Modifications to `aff-panel-left.js`

Add `renderClassesTree(categories, classCounts)` function following the variable tree renderer. Classes count badges are populated from `AFF.Utills.getClassesForCategoryId()`.

8.4 Load Order

`aff-classes.js` loads after `aff-app.js` (which defines `AFF.CatMixin`) and before the `DOMContentLoaded` init sequence runs. Add to the enqueue dependency chain in the PHP admin enqueue handler in `class-aff-admin.php`.

9. CSS Implementation Plan

9.1 New File: `admin/css/aff-classes.css`

A new CSS file for class-specific styles. Follows the pattern of `aff-variables.css` (which handles Fonts/Numbers).

Contents:

- `.aff-class-row` — row layout (reuses `.aff-color-row` grid structure; minor overrides for the different column set)
- `.aff-class-name` — monospace, inherits from variable name styles
- `.aff-class-category-badge` — small badge; can reuse the existing category badge style if one exists, otherwise new
- `.aff-class-styles-flag` — `has styles` / `no styles` indicator with muted color for `no styles`
- `.aff-classes-empty-state` — centered empty-state message with action buttons

What can be reused from `aff-colors.css`:

- `.aff-color-row` base structure
- `.aff-drag-handle` styles
- `.aff-status-dot` and status color tokens (`--aff-status-synced`, `--aff-status-modified`, etc.) — already defined in `aff-theme.css`
- `.aff-category-block` structure
- `.aff-icon-btn` styles

What is new:

- Column grid definition for the class list (5 columns: handle + name + category + status + actions; no swatch/preview/value columns)
- Status badges for `aff-only` and `orphaned` states (colors exist as CSS vars in `aff-theme.css`; just need selectors)

9.2 Enqueue

Add `aff-classes.css` to the admin enqueue list in `class-aff-admin.php`, after `aff-variables.css`.

10. Phased Delivery

Phase 3.1 — Foundation (Data Layer)

Goal: `aff_sync_classes` AJAX action works end-to-end.

Files to create/modify:

- **Create** `includes/class-aff-classes-reader.php`
- **Modify** `includes/class-aff-data-store.php` — add `classCategories` to map, add Classes CRUD methods, `class_defaults()`, migration block
- **Modify** `includes/class-aff-ajax-handler.php` — add `aff_sync_classes` and `aff_get_classes` endpoints
- **Modify** `includes/class-aff-loader.php` — register new file

Deliverable: Calling `aff_sync_classes` returns a normalized class list from Elementor and stores it in the project file.

Verification: Can be confirmed by calling the AJAX action directly via browser DevTools and inspecting the response + the `.aff.json` file.

Phase 3.2 — Left Panel + Basic List View

Goal: Classes are navigable in the UI.

Files to create/modify:

- **Modify** `admin/js/aff-panel-left.js` — add `renderClassesTree()`
- **Create** `admin/js/aff-classes.js` — `init()`, `loadClasses()`, basic `_renderView()` with read-only rows

- **Create** `admin/css/aff-classes.css` — row grid, badges, empty state
- **Modify** `admin/js/aff-app.js` — wire `AFF.Classes.init()`, add `findClassById()` / `getClassesForCategoryId()` to `AFF.Utils`
- **Modify** `class-aff-admin.php` — enqueue new JS/CSS

Deliverable: ► `Classes` in the left panel expands to show categories with counts. Clicking a category shows the class list. Sync button works. Status badges render correctly.

Phase 3.3 — Detail Modal + Category Management

Goal: Full metadata editing, category CRUD.

Files to modify:

- **Modify** `admin/js/aff-classes.js` — add `_openDetailModal()`, tag editing, notes, category dropdown
- **Modify** `includes/class-aff-ajax-handler.php` — add `aff_update_class_meta`, `aff_save_class_category`, `aff_delete_class_category`, `aff_reorder_class_categories`

Reuse: `AFF.CatMixin` handles add/rename/delete/reorder category operations with zero new code.

Deliverable: Users can categorize classes, add notes and tags, reorder categories, and drag-reorder classes within a category.

Phase 3.4 — Elementor Write Operations (Create / Rename / Delete)

Goal: AFF can create, rename, and delete Global Classes in Elementor.

Files to create/modify:

- **Create** `includes/class-aff-classes-writer.php`
- **Modify** `includes/class-aff-ajax-handler.php` — add `aff_create_class`, `aff_rename_class`, `aff_delete_class`
- **Modify** `admin/js/aff-classes.js` — wire create/rename/delete UI flows

Deliverable: Users can create empty Global Classes from AFF (no styles — must add styles in Elementor editor), rename existing classes, and delete classes — all synced to Elementor via the REST PUT endpoint.

Risk: Rename and delete are destructive in Elementor. The UI must show a confirmation modal with a clear warning that the change is immediate and affects the live Elementor site.

11. Open Questions

11.1 (RESOLVED) Meta key name

Confirmed: `_elementor_global_classes` (frontend) and `_elementor_global_classes_preview` (editor preview).

Source: `modules/global-classes/global-classes-repository.php`, lines 15–16.

11.2 (RESOLVED) Storage structure

Confirmed: `{ items: { [g-id]: { id, type, label, variants } }, order: [...] }.`

Note: REST response wraps this as `{ data: ..., meta: { order: ... } }` — different from stored shape.

Source: `global-classes-repository.php` line 58, `global-classes-rest-api.php` lines 148–155.

11.3 (RESOLVED) Class limit

Confirmed: 100 (raised from earlier 50 in Alpha).

Source: `global-classes-rest-api.php` line 18: `const MAX_ITEMS = 100.`

11.4 (RESOLVED) Variant structure

Confirmed: `{ props: { [css-prop]: $$type-value }, meta: { breakpoint, state }, custom_css: { raw } }.`

Source: `styles-renderer.php` lines 116–136, `classes-provider.php` lines 58–85.

11.5 (RESOLVED) REST authentication

Confirmed: Standard `wp_rest` nonce for GET. Custom capability

`elementor_global_classes_update_class` for PUT — administrator role only by default.

Source: `global-classes-rest-api.php` lines 38–42 and 77, `database/migrations/add-capabilities.php` line 8.

11.6 (OPEN) Feature flag status on the main `site` local install

The Global Classes module requires both `e_classes` AND `e_atomic_elements` experiments to be active. On the development site (`site.local`), both must be enabled in Elementor > Settings > Features before Phase 3.1 testing can proceed.

If both are inactive, `get_post_meta($kit_id, '_elementor_global_classes', true)` will return an empty string — indistinguishable from "no classes created yet." AFF should detect this and show a message directing the user to enable the feature.

How to check: Log into `site.local/wp-admin` → Elementor → Settings → Features → confirm `Global Classes` is Active.

11.7 (OPEN) Behavior of `get_json_meta` vs `get_post_meta` on raw value

Elementor's `Kit::get_json_meta()` is a document method that handles deserialization. AFF uses raw `get_post_meta()`. Testing is needed to confirm whether the meta value on a live site is stored as a JSON string or as a PHP-serialized array. If it arrives as an already-decoded array (due to WordPress's auto-unserialization of some serialized formats), the `is_string($raw)` branch in `read_from_postmeta()` may never execute.

How to verify: In `aff_sync_classes` handler, log `gettype(get_post_meta($kit_id, '_elementor_global_classes', true))` on a site that has classes. Should be `string`.

11.8 (OPEN) `sync_to_v3` field handling

The `design-system-sync` module adds a `sync_to_v3` boolean to class items (line 39 of `classes-provider.php`). AFF's normalization must preserve this field verbatim in the stored item data when writing back to Elementor — stripping unknown fields from items would break V3 sync for users who have it enabled.

Resolution: In `AFF_Classes_Writer`, when constructing the `items` payload for the PUT request, merge AFF's changes onto the original Elementor item data rather than building items from scratch.

11.9 (OPEN) CSS selector prefix in rendered output

`Styles_Renderer` uses `.elementor` as a selector prefix (line 8 of `styles-renderer.php`). The rendered class CSS is `.elementor .primary-btn { ... }` — not `.primary-btn { ... }` bare. This matters for any future feature where AFF previews or displays the generated CSS. For Phase 3.1–3.3 (display-only, no CSS rendering), this is informational only.

End of document. All sections are implementation-ready pending resolution of open questions 11.6 and 11.7, which can be confirmed in a single 10-minute testing session on the development site.